

UNIVERSITÉ DE YAOUNDÉ I

Faculté des Sciences

Département d'Informatique



Master I Systèmes et Réseaux

TRAVAIL PRATIQUE — INF4218

PROGRAMMATION DISTRIBUÉE

SYSTÈME DISTRIBUÉ D'APPRENTISSAGE FÉDÉRÉ

MEMBRES DU GROUPE

NOM ET PRÉNOMS	MATRICULE
MFENJOU ANAS CHERIF	21T2330
EDU GUIEDI HERMANN ARNOLD	22T2876
LANGOUL FADILA MARIAMA MOUNIRA	21T2528

ANNÉE ACADÉMIQUE

2025–2026

SUPERVISÉ PAR

Dr ADAMOU HAMZA

1. Introduction

1.1 Contexte et objectifs

Ce projet s'inscrit dans le cadre du cours **INF4218 — Programmation Distribuée**, enseigné au Master I Systèmes et Réseaux de l'Université de Yaoundé I. Il vise à mettre en œuvre les concepts fondamentaux des systèmes distribués vus en cours, d'après l'ouvrage de Van Steen et Tanenbaum (*Distributed Systems*, chapitres 1 à 8).

L'objectif est de concevoir un **système distribué d'apprentissage fédéré** (*Federated Learning*, FL) : plusieurs clients entraînent un modèle localement sur leurs données, envoient leurs gradients à un serveur central qui les agrège pour produire un modèle global. Ce scénario illustre coordination, communication à distance, cohérence des événements et tolérance aux pannes.

1.2 Concepts obligatoires

Concept	Chapitre Tanenbaum
Nommage plat	6.2
Nommage structuré	6.3
Nommage par attributs	6.4
Horloges logiques de Lamport	5.2.1
Élection de leader (Bully)	5.4.1
Exclusion mutuelle distribuée	5.3.3
Communication RPC/gRPC	4
Tolérance aux pannes (checkpoint)	8.6.2

1.3 Technologies

Python 3.14, gRPC/Protocol Buffers, NumPy (simulation des gradients), pytest (38 tests unitaires + 1 intégration). Architecture client-serveur à **trois niveaux** : application, middleware, réseau.

2. Conception

2.1 Architecture globale

Le système suit une architecture en trois couches inspirée du chapitre 2 :

```
+-----+
|  COUCHE APPLICATION  |
```

```

| Rounds FL, entraînement local, agrégation des gradients|
+-----+
| COUCHE MIDDLEWARE |
| Lamport, Bully, mutex, nommage (3 types), checkpoint |
+-----+
| COUCHE RÉSEAU (gRPC) |
| RPC unary + streaming (gradients et modèle global) |
+-----+

```

Serveur coordinateur (`FederationCoordinator`, process 0) : coordonne les rounds, maintient le modèle global, gère les registres de nommage, applique Lamport/Bully/mutex, effectue les checkpoints.

Clients fédérés (`FederatedClient`) : s'enregistrent via les trois nommages, entraînent localement, envoient/reçoivent gradients et modèle, envoient des heartbeats.

2.2 Flux d'un round

Client	Coordinateur
---- Heartbeat (Lamport ts) ---->	
<--- Round actuel + ts -----	
[Entraînement local]	
---- UploadGradients (stream) --->	[Mutex : agrégation]
<--- DownloadModel (stream) -----	
[Checkpoint si round % 3 == 0]	

2.3 Systèmes de nommage

Plat (UUID) : identifiant opaque unique par client, résolution $O(1)$ dans un registre central.

Structuré : chemins hiérarchiques, ex. `/federation/clients/client1/models/local`, inspirés du DNS et des systèmes de fichiers Unix.

Par attributs : recherche par paires clé-valeur (`dataset: iris, location: local`) via `QueryByAttributes`, permettant une sélection dynamique des participants.

2.4 Mécanismes de coordination

Lamport : chaque processus maintient un compteur C_i . Avant un événement local : $C_i \leftarrow C_i + 1$. À la réception d'un message de timestamp ts : $C_j \leftarrow \max(C_j, ts) + 1$. Garantit : si $a \rightarrow b$ (happens-before), alors $C(a) < C(b)$.

Bully : le processus d'ID maximal vivant devient coordinateur. En cas de panne, le détecteur envoie ELECTION aux IDs supérieurs ; le gagnant annonce COORDINATOR.

Ricart-Agrawala : l'agrégation est une section critique protégée par mutex distribué. Le plus petit timestamp Lamport accède en premier ; pas de deadlock ni de famine.

Checkpoint : sauvegarde JSON tous les 3 rounds (round, poids, horloge Lamport). Recovery automatique au redémarrage du serveur.

2.5 Contribution des concepts au système fédéré

Chaque concept apporte une garantie concrète. Le tableau résume son **rôle**, son **apport** et le **comportement sans lui**.

Concept	Rôle dans le FL	Apport	Sans ce concept
Architecture 3 niveaux	Sépare entraînement, coordination, transport	Modules testables ; évolution facilitée	Monolithe fragile, maintenance difficile
Nommage plat	UUID client (<code>flat_id</code>)	Identité unique ; indépendant de l'IP	Collisions, écrasement de gradients
Nommage structuré	Chemin hiérarchique	Annuaire administrable (type DNS)	Registre plat ingérable
Nommage par attributs	<code>{dataset, location}</code>	Sélection dynamique des clients	Liste statique, pas de client sampling
Lamport	Timestamp dans chaque message gRPC	Ordre causal des rounds ; base du mutex	Mélange de rounds, incohérence du modèle
Mutex Ricart-Agrawala	Protège <code>aggregate_round()</code>	Intégrité de <code>model_weights</code>	Corruption par agrégations concurrentes
Bully	Élection du coordinateur	Continuité de service après panne	Arrêt total, reconfiguration manuelle

Concept	Rôle dans le FL	Apport	Sans ce concept
gRPC	Canal client-serveur unique	Contrat typé ; unary + streaming	Formats ad hoc, pas d'interop
Checkpoint	Sauvegarde périodique	Reprise rapide ; perte max. 2 rounds	Perte totale du progrès à chaque crash

Apports détaillés par concept :

- **Architecture** : la couche middleware centralise Lamport, Bully et le mutex ; la couche réseau isole gRPC. On modifie la communication sans toucher à l'algorithme d'agrégation.
- **Nommage plat** : à l'inscription, chaque client reçoit un UUID. Ce nom lie les gradients reçus, les heartbeats et les entrées de checkpoint. Sans lui, l'IP ou le `process_id` seul provoquerait des collisions derrière un NAT.
- **Nommage structuré** : le chemin `/federation/clients/clientN/models/local` permet de naviguer dans l'espace de noms comme un annuaire. Sans lui, retrouver le modèle local d'un client exigerait un parcours complet du registre.
- **Nommage par attributs** : le coordinateur peut inviter uniquement les clients `{dataset: mnist}` pour un round cible. Sans lui, seule une configuration statique serait possible.
- **Lamport** : chaque RPC (heartbeat, gradient, mutex) transporte un timestamp. Les événements de rounds sont ordonnés dans `round_events`. Sans Lamport, deux clients envoyant simultanément n'auraient aucun ordre défini : risque de mélanger des gradients de rounds différents.
- **Mutex** : `aggregate_round()` acquiert le verrou avant la moyenne des gradients. Sans mutex, deux threads d'agrégation corrompraient `model_weights`.
- **Bully** : si le coordinateur tombe, le processus vivant d'ID maximal prend le relais via `SendElection` / `AnnounceCoordinator`. Sans Bully, les clients enverraient des heartbeats dans le vide indéfiniment.
- **gRPC** : `federation.proto` définit 12 RPC. Unary pour messages légers ; streaming par chunks de 4 Ko pour gradients et modèle (seuil 8 Ko). Sans gRPC structuré, chaque composant inventerait son propre format binaire.
- **Checkpoint** : `_recover_state()` recharge `checkpoints/latest.json` au démarrage. Sans checkpoint, chaque crash du serveur remettrait le round à zéro et réinitialiserait le modèle global.

3. Détails d'implémentation

3.1 Structure du projet

TPSD/

```
proto/federation.proto      -- Service gRPC (12 RPC)
src/naming/                 -- flat.py, structured.py, attribute.py
src/coordination/           -- lamport.py, bully.py, mutual_exclusion.py
src/fault_tolerance/        -- checkpoint.py
src/server/coordinator.py   -- Serveur coordinateur
src/client/federated_client.py
tests/                      -- 38 tests unitaires
experiments/benchmarks.py
```

3.2 Communication gRPC

Le service FederationCoordinator expose :

- **Unary** : RegisterClient, Heartbeat, RequestMutualExclusion, ReleaseMutualExclusion, ResolveName, QueryByAttributes, SendElection, AnnounceCoordinator.
- **Streaming** : UploadGradients (client vers serveur), DownloadModel (serveur vers client).

Chaque message transporte un `lamport_timestamp`. L'inscription enregistre simultanément les trois nommages :

```
self.flat_naming.register(address, flat_id=flat_id)
self.structured_naming.register(request.structured_path, address)
self.attribute_naming.register(flat_id, address, dict(request.attributes))
```

3.3 Modules de coordination

Lamport (`lamport.py`) : `tick()` pour événements locaux, `send_event()` à l'envoi, `receive_event(ts)` à la réception. Tie-breaker par (`time`, `process_id`).

Bully (`bully.py`) : `start_election()`, `handle_election_message()`, `mark_coordinator_dead()`. Callbacks injectés pour découpler la logique de la communication gRPC.

Mutex (`mutual_exclusion.py`) : `request_access()` diffuse une demande horodatée; `handle_request()` accorde ou diffère selon le timestamp; `release_access()` envoie les OK différés.

Agrégation protégée (`coordinator.py`) :

```

def aggregate_round(self):
    if not self.mutex.request_access("aggregation"):
        return
    try:
        self.model_weights = np.mean(all_grads, axis=0).tolist()
        self.current_round += 1
    finally:
        self.mutex.release_access("aggregation")

```

3.4 Tolérance aux pannes

SystemState sauvegarde : current_round, coordinator_id, lamport_clock, model_weights, round_history. Intervalle : tous les 3 rounds. Détection de panne par timeout heartbeat (5 s).

3.5 Tests

Module	Tests	Couverture
Lamport	7	tick, send/receive, happens-before
Bully	7	élection, panne, annonce coordinateur
Nommage	12	plat, structuré, attributs
Mutex	5	accès, conflit, release
Checkpoint	7	save/load, recovery, timeout

Commande : `pytest tests/ -v` — 38 passed.

4. Résultats expérimentaux

Expériences exécutées via `python experiments/benchmarks.py`.

4.1 Cohérence Lamport

Protocole : 5 processus, 10 rounds ; chaque processus envoie un message horodaté à tous les autres.

Vérification : `C(réception) > C(envoi)` pour chaque paire.

Métrique	Valeur
Processus / Rounds	5 / 10
Paires vérifiées	200

Métrique	Valeur
Violations	0
Cohérence	Oui

Les horloges Lamport garantissent l'ordonnancement causal des événements de rounds, condition nécessaire à une agrégation fiable.

4.2 Streaming vs unary (gRPC)

Protocole : buffers de 1 Ko à 256 Ko ; comparaison unary vs streaming (chunks 4 Ko).

Taille (o)	Chunks	Unary (ms)	Streaming (ms)	Ratio
1 024	1	0.001	0.006	10,8×
16 384	4	0.000	0.006	20,9×
262 144	64	0.000	0.041	83,9×

Unary préféré pour messages < 8 Ko ; streaming avantageux pour gros modèles (robustesse, pipelining réseau).

4.3 Élection Bully

Protocole : 8 processus, panne de P7, élection déclenchée par P4.

Métrique	Valeur
Coordinateur initial	P7
Nouveau coordinateur	P6
Temps d'élection	0,17 ms

P4 envoie ELECTION à P5 et P6 ; P6 (ID maximal vivant) devient coordinateur et annonce sa victoire.

4.4 Checkpoint / recovery

Protocole : 9 rounds simulés, checkpoints aux rounds 3/6/9, crash simulé (timeout 2 s).

Métrique	Valeur
Round récupéré	9

Métrique	Valeur
Crash détecté	Oui
Poids restaurés	$[0,9 \times 10]$

Reprise exacte au dernier état cohérent ; perte maximale de 2 rounds entre deux checkpoints.

4.5 Synthèse

Expérience	Résultat
Lamport	0 violation / 200 paires
gRPC	Unary < 8 Ko ; streaming > 64 Ko
Bully	P6 élu en 0,17 ms
Checkpoint	Round 9 restauré

5. Conclusion

5.1 Bilan

Ce projet implémente un système d'apprentissage fédéré intégrant l'ensemble des concepts INF4218. L'architecture en trois niveaux sépare clairement les responsabilités. Chaque concept distribué apporte une garantie mesurable : ordre causal (Lamport), intégrité (mutex), continuité (Bully), interopérabilité (gRPC), persistance (checkpoint). Les 38 tests et 4 expériences valident le comportement attendu.

5.2 Limitations et perspectives

Limitations : coordinateur central (goulot d'étranglement), Bully $O(n^2)$ en messages, gRPC non chiffré, clients homogènes Python, pas de tolérance Byzantine.

Perspectives : remplacer Bully par Raft, activer TLS/mTLS, Secure Aggregation, clients multilingages via gRPC, compression de gradients, réplication du coordinateur.

6. Références

1. Van Steen M., Tanenbaum A.S., *Distributed Systems*, 4th edition, 2024.
2. Lamport L., « Time, Clocks, and the Ordering of Events in a Distributed System », *Communications of the ACM*, 21(7), 1978.

3. Garcia-Molina H., « Elections in a Distributed Computing System », *IEEE Transactions on Computers*, 31(1), 1982.
4. Ricart G., Agrawala A.K., « An Optimal Algorithm for Mutual Exclusion in Computer Networks », *Communications of the ACM*, 24(1), 1981.
5. McMahan B. et al., « Communication-Efficient Learning of Deep Networks from Decentralized Data », *AISTATS*, 2017.
6. Documentation gRPC : <https://grpc.io/docs/languages/python/>

Reproductibilité : voir README.md (pytest tests/ -v, ./run_demo.sh).